

Cahier des Charges

1. Contexte et Objectif

Les associations étudiantes manipulent des documents sensibles (comptes-rendus, bilans financiers) contenant des Informations Personnelles Identifiables (PII) d'anciens élèves (noms, emails, téléphones, IBAN). L'objectif de ce projet est de fournir un outil automatisé permettant d'anonymiser massivement ces archives en respectant le RGPD, afin de pouvoir les conserver et les transmettre aux nouveaux mandats sans risque de fuite de données privées.

2. Stack Technique Globale

- **Backend (API & Worker)** : Python 3.12+, FastAPI
- **Moteur NLP / Anonymisation** : Microsoft Presidio, spaCy (fr_core_news_md), PyMuPDF (PDF)
- **File d'attente & Cache** : Redis
- **Frontend** : React (Vite), TailwindCSS
- **Infrastructure** : Docker & Docker Compose (Prêt pour Kubernetes en production)



Niveau 0 : Preuve de Concept (PoC) NLP

Objectif : Valider le moteur d'intelligence artificielle et les règles de détection sur du texte brut en français.

Fonctionnalités attendues

- Détection par IA (spaCy) : Noms de personnes (PERSON), Lieux (LOCATION), Organisations (ORGANIZATION).
- Détection par Règles (Presidio/Regex) : Emails, Numéros de téléphone, IBAN, Cartes Bancaires.
- Remplacement des PII par des balises contextuelles (ex: [NOM], [IBAN]).

Implémentation technique

- Script Python standalone (sans serveur).
- Configuration de presidio-analyzer avec le modèle fr_core_news_md pour une meilleure précision sur le contexte français avec un modèle léger.
- Définition des OperatorConfig dans presidio-anonymizer pour personnaliser le texte de remplacement.



Niveau 1 : Minimum Viable Product (MVP) Web

Objectif : Fournir une interface web utilisable par les membres de l'association pour anonymiser des fichiers textes standards de manière asynchrone.

Fonctionnalités Frontend (React)

- **Zone de Drag & Drop** : Permettre l'upload d'un ou plusieurs fichiers (.txt, .md, .docx).
- **Liste Blanche** : Permettre d'ajouter des mots à ne pas censurer (jargon de l'asso).
- **Tableau de bord de file d'attente** : Affichage en temps réel du statut de chaque fichier (En attente, En cours, Terminé, Erreur, Téléchargé (par l'utilisateur)) et de la position dans la file.
- **Bouton de téléchargement** : Permettre de récupérer le fichier anonymisé (gardant le même format que l'original).
- *Optionnel* : Paramètres permettant à l'utilisateur de cocher/décocher ce qu'il veut masquer (Noms, IBAN, Emails...). (tous activés par défaut)

Fonctionnalités Backend (FastAPI + Redis)

- **API Gateway** : Route d'upload (POST /upload) qui sauvegarde le fichier sur disque et ajoute un task_id dans Redis.
- **Statut (Polling)** : Route (GET /status/{task_id}) appelée régulièrement par React pour vérifier l'avancement.
- **Gestion .docx** : Lecture et réécriture des paragraphes avec la librairie python-docx en conservant la structure.
- **Worker Asynchrone** : Script Python tournant en boucle, qui "dépile" les tâches Redis une par une pour ne pas surcharger la RAM/CPU du serveur.

La limite de nombre de workers est dans le fichier de config (avec une précision dans ce fichier de la quantité de RAM consommée par worker).

Limites de ce niveau

- Le système de fichiers est local (les dossiers uploads/ et processed/ sont sur le serveur).
- Pas de prise en charge des PDF.

Niveau 2 : Prise en charge des Formats Complexes (PDF & Slides)

Objectif : Étendre l'outil aux formats de présentation et d'impression, qui constituent la majorité des archives.

Fonctionnalités

- Support des fichiers .pdf.
- Censure visuelle (Redaction) : Les mots sensibles ne sont pas "remplacés" (ce qui casserait la mise en page), mais recouverts par des rectangles noirs inamovibles, et le texte sous-jacent est détruit.

Implémentation technique

- **Intégration de PyMuPDF (fitz) dans le Worker :**
 1. Extraction du texte de chaque page.
 2. Analyse par Presidio pour obtenir les mots à censurer.
 3. Recherche des coordonnées (X,Y) de ces mots sur la page PDF.
 4. Application d'annotations de censure (`add_redact_annot`) et validation (`apply_redactions`).
- **Gestion .pptx (Optionnelle selon le temps) :** Utilisation de `python-pptx` pour itérer à travers les "shapes" (zones de texte) de chaque slide, similaire à l'approche `.docx`.



Niveau 3 : Automatisation Cloud (Bot Google Drive)

Objectif : Permettre un traitement par lots massif "pendant la nuit" sans aucune intervention manuelle, en se branchant directement sur le Drive de l'association.

Fonctionnalités

- **Bot Service Account :** L'utilisateur partage un dossier Google Drive avec l'adresse email d'un bot (ex: `anonymizer-bot@asso-project.iam.gserviceaccount.com`).
- **Mode "Non-Destructif" (Copie) :** Le bot ne supprime/modifie **jamais** les fichiers originaux. Il crée de nouveaux fichiers à côté de l'original avec le même nom (juste [ANONYMIZED] à la fin du nom).
- **Rapport de Traitement :** Génération d'un fichier `.csv` et `.md` (csv désactivé par défaut mais on peut le cocher dans le front) à la racine du dossier listant : les fichiers traités, les fichiers en erreur, et le nombre d'entités trouvées par fichier. Ces rapports sont mis à jour à chaque fichiers anonymisés.

Implémentation technique

- **Google Drive API :** Utilisation de la librairie d'authentification serveur-à-serveur (Service Account Keys) de Google Cloud Console.
- **Worker Drive Spécialisé :** Un nouveau type de worker Redis (ou une tâche CRON) qui prend un `folder_id` en paramètre, liste les fichiers, les télécharge, les passe dans le Worker d'anonymisation local, puis les ré-uploade.
- **Gestion des quotas :** Ajout de délais (sleep) dans le script pour respecter les limites de l'API Google (Rate Limiting).

4. Règles de Sécurité et RGPD (Transverse)

- **Volatilité des données :** Aucun fichier uploadé par un utilisateur via le portail web (Niveau 1 et 2) ne doit être conservé. Un script (ou un conteneur dédié) devra purger les dossiers `uploads/` et `processed/` toutes les heures (attention uniquement les fichiers déjà téléchargés (sinon s'ils sont là depuis plus de 24h)).

- **Logs** : Les logs du Worker ne doivent **jamais** imprimer les noms ou entités trouvés dans la console (risque de fuite dans les systèmes de monitoring).
- **Isolement** : Le modèle NLP tourne entièrement en local sur le cluster Kubernetes/Docker de l'école. Aucune donnée n'est envoyée à une API externe type OpenAI.